SeedingMinds

Assessment Name

Java Coding_S1_100196

Module: Coding

Question 1:

Q1 - Read Array Size and Elements

Write a Program to read the Array Size and Array Values from the User and Print Array Values?

Constraints:

Input :- First Line of Input Consists of One Integer Value (Array Size).

Second Line of Input Consists of few Integer Values Separated by Spaces (Array Elements).

Output :- Print the All Array Elements.

Constraints :- **No**

Example:

Input 1 : 8

86 210 33 73 53 93 848 48

Output 1: 86 210 33 73 53 93 848 48

Input 2 : 6

1 2 3 4 5 6

Output 2: 1 2 3 4 5 6

Input 3 : 5

5 10 88 2 5

Output 3: 5 10 88 2 5

Explanation:

NA

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	8 86 210 33 73 53 93 848 48	86 210 33 73 53 93 848 48
2	6 1 2 3 4 5 6	1 2 3 4 5 6
3	5 5 10 88 2 5	5 10 88 2 5

Question 2:

Password Validation

Write a program to check/validate given password is perfect or not ?

Constraints:

Input must be a string and it should contain **9 characters** or else Print **Not a Perfect Password.**

Atleast one character should be **small** or else Print **Not a Perfect Password.**

Atleast one character should be **capital** or else Print **Not a Perfect Password.**

Password should contain atleast **one Number** or else Print **Not a Perfect Password.**

Password should contain atleast **one Special Character**(Except Space) or else Print **Not a Perfect Password.**

Example:

Input 1 : abCde@123

Output 1 : Perfect Password

Input 2 : 0123456789

Output 2 : Not a Perfect Password

Explanation:

NA

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	abCde@123	Perfect Password
2	0123456789	Not a Perfect Password

Question 3:

Early Exit Optimization

In a world where data drives decisions, an e-commerce company constantly seeks ways to optimize its processes to enhance customer experience. One critical aspect of their operations is sorting product listings based on various criteria, such as price, ratings, or popularity. The existing sorting algorithm is bubble sort, which, while easy to implement, struggles with efficiency, especially when handling large datasets during peak shopping seasons.To address this issue, the company decides to implement an optimization strategy for bubble sort that reduces unnecessary iterations. The optimization is based on the observation that bubble sort can exit early if no swaps occur during a complete pass through the array. This indicates that the array is already sorted, allowing the algorithm to terminate early instead of continuing to iterate through the remaining elements.

Constraints:

- The array can contain at most 10^3 integers.
- Each integer in the array can be in the range of 0≤price≤10^6

Example:

Example 1:

Input: 5
5 2 9 1 5
Output: 1 2 5 5 9
4

Example 2:

Input: 3
3 4 5
Output: 3 4 5
1

Explanation:

Your task is to implement bubbleSort() function with early exit optimization in Java. The goal is to ensure that the algorithm exits as soon as the array is sorted, avoiding unnecessary iterations. You will need to track and display the number of iterations taken until the array becomes sorted to demonstrate the effectiveness of the optimization.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	7 11 4 7 3 1 9 6	1 3 4 6 7 9 11 5
2	4 7 6 5 4	4 5 6 7 4

Question 4:

Neural Grid Optimization

In a futuristic world where artificial intelligence governs every facet of life, there exists a highly
advanced computational structure known as the Neural Grid. This grid is responsible for processing data
flowing through various nodes represented in a matrix. Each diagonal in this matrix plays a crucial role in
the stability and efficiency of the system. However, over time, the values in the grid start to fluctuate,
causing instability in the AI's decision-making processes.
You are assigned as the chief data architect responsible for restoring balance in the Neural Grid. To achieve
this, you must optimize each diagonal of the matrix by sorting it in ascending order. Diagonals in the Neural
Grid are lines of nodes that start from any node in the top row or leftmost column and move in a bottom-right
direction.
Given the current state of the grid as an m x n matrix, your task is to sort each diagonal independently, while
ensuring that the system remains optimized for high performance.

Constraints:

- m == grid.length
- n == grid[i].length

- 1 <= m, n <= 100
- 1 <= grid[i][j] <= 100

Example:

Example 1:

Input: 5 4 3
2 1 6
9 8 7
end
Output: [1, 4, 3]
[2, 5, 6]
[9, 8, 7]

Example 2:

Input: 3 1 2
4 6 5
9 7 8
end

Output: [3, 1, 2]
[4, 6, 5]
[9, 7, 8]

Explanation:

The task of the problem involves processing a given matrix of integers, where the goal is to
sort the values along each diagonal in ascending order while maintaining the overall structure of the matrix.
A diagonal in this context is defined as a sequence of elements that starts from any cell in the topmost row
or leftmost column and extends downward to the right until reaching the matrix's boundary. After sorting the
diagonals, the transformed matrix is returned, reflecting the updated values in their respective positions. The
challenge lies in efficiently sorting these diagonals while ensuring that the input matrix is correctly modified
according to the sorting criteria. This problem not only tests one's ability to manipulate multi-dimensional
arrays but also emphasizes understanding diagonal relationships within a matrix.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	15 9 3 4 1 2 6 7 5 end	[1, 2, 3] [4, 5, 9] [6, 7, 15]
2	12 11 10 1 7 6 5 2 8 3 4 9 end	[4, 5, 2, 1] [3, 6, 9, 10] [8, 7, 12, 11]